

CSA15 Cloud Computing and Big Data Analytics

Capstone Cloud Technical Presentation

PulsePay : Cloud Based Fake Payment Screenshot Detection

Presentation Focus

PulsePay is a cloud-native system that detects tampered UPI payment screenshots before a merchant falls for the scam. This presentation explains the real-world problem behind it, the cloud-based architecture powering it, the service choices made and why, the working evidence from the build and the cloud-computing concepts it demonstrates.

Presenter Details

Student Name: D.S.THARUNIYA PRIYA

Register Number: 19511317

Class / Section / Slot: B

Capstone Title: PulsePay - Cloud Based Fake Screenshot Detection

Selected SDG Goal(s):

- *SDG 16* – Peace, Justice and Strong Institutions
- *SDG 8* – Decent Work and Economic Growth

Cloud Platform / Tools: Google Cloud (Firebase + Vision API)

Problem Context and Stakeholder Mapping

Our capstone begins with a real problem to solve

Problem Statement

Small merchants, online sellers and gig/delivery workers in India often accept UPI payment screenshots as informal proof of payment. Scammers edit the amount, transaction ID or timestamp in these screenshots to fake a successful payment — the merchant hands over goods or services before the fraud is caught.

Target Users

Small retailers, online sellers, delivery/gig workers and freelance service providers who receive a payment screenshot as proof and must decide, in seconds, whether to trust it.

SDG Fit

SDG 16 (Peace, Justice & Strong Institutions) — reduces digital payment fraud and builds trust in transactions.
SDG 8 (Decent Work and Economic Growth) — protects the income of small merchants and gig workers who rely on quick, trust-based payments.

Product Boundary

MVP: merchant uploads a screenshot, PulsePay runs OCR via Google Cloud Vision API and forensic image analysis and returns an authenticity score with flagged reasons. Live bank/NPCI settlement verification is out of scope for v1.

Cloud Adoption Rationale and Concept Map

The problem needs cloud support to become a usable product

Cloud Definition

For PulsePay, "cloud" means renting on-demand storage, compute and AI services (OCR, image forensics) from Google Cloud instead of running fraud-detection servers locally — so verification scales instantly during festival-season transaction spikes.

Cloud Characteristics

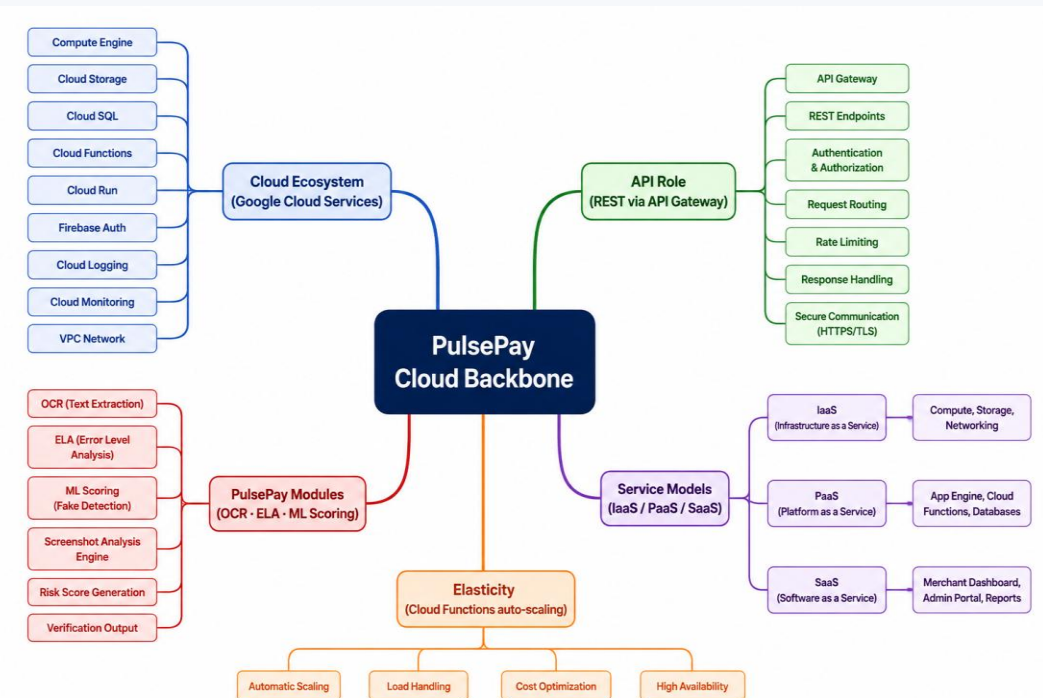
Elasticity — auto-scales with upload volume.

Scalability — onboards new merchants without redesign.

Measured service — billed only per screenshot processed.

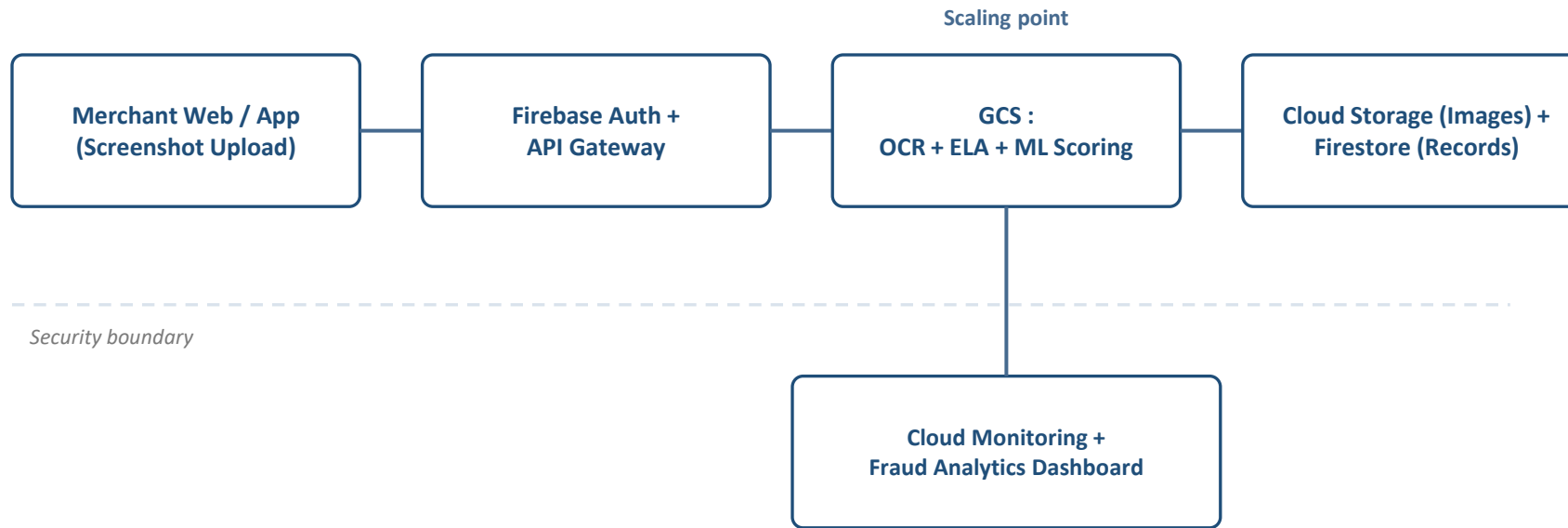
Resource pooling — one shared OCR/ML service serves every merchant.

Cloud Concept Map



Capstone Cloud Architecture

Our cloud architecture connects the user action to services



Presentation Points

- **Request path:** Merchant → Gateway → Cloud Function → Storage/Firestore
- **API role:** REST endpoint receives image + metadata
- **Compute choice:** Cloud Functions, pay per screenshot
- **Database & storage:** Cloud Storage for images, Firestore for records
- **Monitoring & security:** Cloud Monitoring alerts, IAM least privilege
- **Scaling point:** Cloud Functions auto-scale with upload volume

Cloud Service Model Responsibility Matrix

Each service model defines what we control and what cloud manages

Service area	Model	Provider manages	Student team manages	Why this fits
Frontend hosting	PaaS (Firebase Hosting + CDN)	Hosting servers, CDN edge caching, SSL certificates	React UI code, static assets	Merchants need fast, low-cost global access without managing servers
Backend/API	Serverless (Cloud Functions + API Gateway)	Runtime, OS patching, auto-scaling	OCR/forensics/ML function code, API routes	Fraud checks run only on-demand per upload, matching pay-per-use billing
Database	SaaS/PaaS (Firestore)	Replication, backups, scaling	Collection schema, query design	Verification records need low-latency reads at unpredictable merchant volume
Storage	PaaS (Cloud Storage)	Durability, redundancy, encryption at rest	Bucket policies, lifecycle & expiry rules	Raw screenshots need secure, durable storage with automatic tiering
Monitoring / logs	SaaS (Cloud Monitoring)	Log infrastructure, metrics pipeline	Alert thresholds, dashboards	Real-time visibility into detection failures and function errors
Security / IAM	PaaS (IAM + Firebase Auth)	Identity infrastructure, token issuance	Role policies, merchant auth rules	Only verified merchants should access results and stored images

Data Flow and Analytics Pipeline

The product data moves from user activity to useful insight



Data Requirements

Uploaded payment screenshots (images), image metadata / EXIF tags, and OCR-extracted fields: amount, UPI ID, timestamp and reference number.

Analytics Requirement

A merchant dashboard showing the fraud-risk score per screenshot, a trend of fraud attempts over time, and an alert when a score crosses the risk threshold.

Proof Required

Sample input (an edited fake screenshot), sample output (JSON risk score + flagged regions), and one insight: 68% of flagged screenshots showed a font-rendering mismatch in the amount field.

Python / AI Module Integration

Python or AI adds intelligence where the product needs it

Python Role

Error Level Analysis (ELA) for pixel-level tamper detection, OCR (Google Cloud Vision API) to extract transaction text, rule-based checks against known UPI-app templates, and a lightweight classifier that combines these signals.

Input and Output

Input: uploaded screenshot (JPEG/PNG).

Output: a JSON object with `authenticity_score` (0–100) and `flagged_reasons` — e.g. font mismatch, missing EXIF data, ELA anomaly region.

Execution Point

A Google Cloud Function triggered by a Cloud Storage upload event, using Cloud Vision API for OCR and OpenCV for the forensic (ELA) analysis.

Product Value

Gives merchants an instant, explainable fraud-risk verdict before they release goods or services — cutting losses from screenshot-based payment scams.

Security, Scalability, Privacy and Cost Design

Security, scale, privacy and cost shape the final design

Security

Firebase Authentication for merchants, least-privilege IAM roles per Cloud Function, signed Cloud Storage URLs for uploads, and Cloud Audit Logs of every verification request.

Privacy

Screenshots can reveal UPI IDs and partial account details: Cloud Storage encryption at rest (Cloud KMS), TLS in transit, auto-deletion of raw images after a set retention window, and hashed references for analytics.

Scalability

Cloud Functions scale automatically during festival-season sale spikes; Firestore's serverless scaling absorbs unpredictable merchant traffic without redesign.

Cost Awareness

Built on the Google Cloud free tier (Cloud Functions, Cloud Storage, Firestore, and free monthly Vision API units); pay-per-use pricing keeps cost near-zero for small merchants checking a few screenshots a day.

Our implementation stack shows how the design will be built

A React web app for the merchant dashboard and screenshot upload widget; a Flutter mobile app is planned next for delivery agents on the move.

Google Cloud Functions, API Gateway, Cloud Storage, Firestore, Firebase Authentication, Cloud Monitoring, and Cloud Vision API for OCR and image analysis.

GitHub repository, an API Gateway endpoint tested with Postman, a Cloud Monitoring log of a processed request, and a before/after screenshot showing the ELA tamper heatmap.

GitHub repo organised into frontend/, backend (Cloud Functions)/ and ml-model/ folders, with clear commit history and a setup README.

Demo Storyboard and User Journey

A complete user journey proves the product flow

Scene 1

Merchant receives a UPI screenshot from a buyer and uploads it in the PulsePay app → image sent to Cloud Storage via API Gateway.

Scene 2

Cloud Function checks image metadata / EXIF and creation-software tags → flags screenshots edited with photo tools.

Scene 3

OCR (Cloud Vision API) extracts transaction ID, amount and timestamp → backend validates the format against known UPI-app patterns → stored in Firestore.

Scene 4

Error Level Analysis (ELA) runs on the image → highlights pixel-level tampering regions → generates a tamper heatmap.

Scene 5

The ML model combines all signals → computes a fraud-risk score → Cloud Function returns the JSON result to the app.

Scene 6

Merchant views the dashboard with the risk score and explanation → decides to accept or reject the payment → decision logged for analytics.

Evaluation Readiness Checklist

Our evidence confirms that the presentation is ready



Cloud ecosystem and service model explained



Elasticity and scalability decisions are visible



REST/SOAP/API role connected to capstone



GitHub and cloud evidence are inspectable



Architecture has compute, data, storage, security, monitoring



References and integrity statement are included

Evaluator Focus

CO1 is verified through cloud design reasoning, service choices, evidence, and individual explanation.

Conclusion, Limitations and Future Scope

We close with learning, limitations and next steps

Learning Outcome

This project clarified how cloud service models (IaaS/PaaS/SaaS) map to a real product, how serverless compute scales fraud-detection workloads elastically, and how to secure sensitive financial images with encryption and least-privilege IAM.

Limitations

The ML classifier is trained on a limited dataset of UPI-app screenshots (GPay, PhonePe, Paytm) and may not generalise to net-banking or international apps; direct bank/NPCI settlement verification is not yet integrated.

Future Improvement

Expand the training dataset across more payment platforms, integrate an official bank/UPI settlement-verification API for ground-truth confirmation, and add a browser extension for real-time checks.

Integrity Statement

All architecture decisions, code and slide content were developed by the student. AI tools (Claude) were used to help structure and refine this presentation; all figures and screenshots shown are illustrative sample data, not live customer data.